

CSCE 614 COMPUTER ARCHITECTURE · MODULE 01 · LECTURE NOTES

Performance: What to Measure

Why intuition about speed is usually wrong.

MODULE OUTCOMES

Decompose execution time into instruction count, CPI, and clock period.

Explain why MIPS and clock speed are misleading metrics.

Apply Amdahl's law and state what it implies for optimisation effort.

Design a benchmark that measures what you intend it to.

Read basic hardware performance counters.

1 The performance equation

Execution time factors into exactly three terms, and every performance change in existence modifies one of them.

$$\text{time} = \text{instruction count} \times \text{cycles per instruction} \times \text{clock period}$$

Factor	Determined by	Who controls it
Instruction count	The algorithm and the compiler	You, through algorithm choice (CSCE 629) and source structure.
CPI	How the microarchitecture responds to <i>your</i> code: cache behaviour, branch predictability, dependency chains, available parallelism	You, indirectly — and this is what the rest of this course is about.
Clock period	Process technology, power and thermal limits	Essentially nobody, any more. Clock frequency has been roughly flat since 2005 (Module 13).

CPI IS NOT A PROPERTY OF THE PROCESSOR

It is a property of a *program running on* a processor. The same chip will deliver a CPI of 0.3 on one loop and 20 on another.

That is the central idea of the course: the hardware provides mechanisms, and whether your code benefits from them or fights them decides performance far more than the clock on the box.

1.1 Metrics that mislead

Metric	Why it misleads
Clock frequency	Says nothing about work per cycle. A 3 GHz core at CPI 4 is slower than a 2 GHz core at CPI 1. Marketing used this for two decades.
MIPS	Rewards executing <i>more</i> instructions. A compiler that emits twice as many instructions in the same time scores twice as well, which is exactly backwards.
FLOPS	Measures floating-point throughput only. Most real programs are limited by memory bandwidth or latency, so peak FLOPS is a number almost no application approaches.
Synthetic benchmarks	Measure the benchmark. When a benchmark becomes commercially important, vendors optimise for it specifically, and it stops predicting anything else.

The only trustworthy metric is wall-clock time on the workload you actually care about. Everything else is a proxy, and proxies get gamed.

2 Amdahl's law

$$\text{speedup} = 1 / ((1 - f) + f/s)$$

where f is the fraction of original execution time affected by your optimisation and s is how much faster that fraction becomes.

f	s	Speedup	Reading
0.9	∞	10×	Making 90% of the program take <i>zero time</i> yields only 10×
0.5	10	1.82×	A tenfold improvement to half the program is worth less than 2×
0.05	100	1.05×	A hundredfold improvement to 5% of the program is almost invisible.
0.99	100	50×	To get large speedups you must improve nearly everything.

WHY 'PROFILE BEFORE OPTIMISING' IS ARITHMETIC, NOT ADVICE

If a routine accounts for 5% of runtime, your maximum possible gain from optimising it is 5%. No amount of cleverness changes that ceiling.

Programmers' intuitions about where time goes are famously unreliable — the hot spot is routinely somewhere nobody suspected. Measuring first is not diligence; it is the only way to know the ceiling exists before you spend a week under it.

Amdahl's law also bounds parallel speedup, with f as the parallelisable fraction. A 5% serial section limits you to 20× no matter how many cores you add, which is a central constraint in CSCE 735. **Gustafson's** counterargument is that in practice people given a bigger machine solve a bigger problem, so f grows with the machine rather than staying fixed — both observations are correct, and they apply to different situations.

3 Measuring honestly

Most performance measurements are wrong, usually in one of a small number of recognisable ways.

Mistake	What you measured instead	Fix
No warm-up	Cold caches, page faults, lazy symbol binding, JIT compilation.	Run the workload several times before timing.
Result never used	Nothing. A compiler that can prove the result is unused will delete the computation entirely, and you will measure an empty loop.	Consume the result: accumulate it, print a checksum, or use a compiler barrier.
Reporting the mean	The worst interference you happened to encounter.	Report the minimum (for machine capability) or the median (for typical experience). Never the mean.
One input size	A single point, with no information about scaling or about which cache level you were in.	Sweep sizes across several orders of magnitude.
Timer inside the inner loop	Mostly the cost of reading the clock, which is substantial.	Time a batch of iterations and divide.
Thermal throttling	The chip's temperature, not your code.	Interleave variants; monitor frequency; distrust small differences.

Thermal throttling is the one that fools experienced people

Under sustained load a modern processor — especially in a laptop — reduces its clock frequency to stay within power and thermal limits. Reductions of 30–50% are routine.

Run variant A for thirty seconds, then variant B, and B executes on a hotter and therefore slower machine. The resulting bias is systematic, reproducible, and looks exactly like a genuine difference.

Interleave the variants (ABABAB rather than AAABBB), discard early runs, log the actual frequency, and treat differences below about 10% on a laptop as unproven.

3.1 A defensible harness

```
double bench(void (*f)(void*), void* data, int trials) {
    f(data); f(data);           // warm caches, predictors, page tables

    double best = 1e30;
    for (int i = 0; i < trials; i++) {
        uint64_t t0 = rdtsc_serialized();
        f(data);
        uint64_t t1 = rdtsc_serialized();
        if ((double)(t1 - t0) < best) best = t1 - t0;
    }
    return best;                // minimum: interference only adds time
}

// Prevent the optimiser from removing the work:
asm volatile("" : : "r"(result) : "memory");
```

Taking the minimum is the right choice when you want to know what the machine is capable of: every source of interference — interrupts, other processes, migrations — only ever adds time, so the fastest observed run is the closest estimate of the underlying cost. When you instead want to characterise typical behaviour under real conditions, report a median and a tail percentile.

4 Hardware performance counters

Processors contain counters for microarchitectural events, and reading them converts guesswork into diagnosis.

Counter	Tells you
cycles, instructions	IPC = instructions / cycles. Below about 1 means the core is stalling; modern cores can sustain 3–4 on good code.
cache-misses, LLC-load-misses	Whether you are memory-bound. A high miss rate with low IPC is the classic memory-bound signature.
branch-misses	Control-flow trouble. Compare against branches for a rate; above ~2% is worth investigating (Module 04).
stalled-cycles-frontend / -backend	Which end is the bottleneck: instruction supply, or execution and memory.
page-faults, dTLB-load-misses	Virtual memory trouble (Module 07).

On Linux, `perf stat ./program` gives most of these in one line. On Windows, Intel VTune and AMD uProf provide equivalents. Learn to read IPC first: it answers the only question that matters at the start, which is whether the core is computing or waiting.

5 The numbers worth memorising

Operation	Approximate cycles	Relative to an ALU op
Register-to-register ALU operation	1	1×
L1 data cache hit	4	4×
L2 cache hit	12	12×
L3 cache hit	40	40×
Main memory (DRAM)	200–300	~250×
Branch misprediction	15–20	~17×
Integer division	20–40	~30×
SSD read	~100,000	100,000×
Network round trip (same datacentre)	~500,000	500,000×

THE ONE COMPARISON THAT ORGANISES THE WHOLE COURSE

An L1 hit is 4 cycles. A main-memory access is around 250. That is a ratio of more than sixty.

The RAM model of CSCE 629 charges **1** for both.

Almost every architectural mechanism in the remaining twelve modules exists either to avoid paying the 250, or to find useful work to do while paying it. Caches, prefetchers, out-of-order execution, speculation, hyperthreading, and the entire design of GPUs are all responses to this one number.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Onur Mutlu — Computer Architecture, Lecture 1–2 (Introduction, Performance)	The framing of the course and the performance equation, done thoroughly.
MIT 6.004 — Performance Measures	The equation and Amdahl's law at an accessible level.
Latency Numbers Every Programmer Should Know	The table from §5 with more entries. Memorise the orders of magnitude.
Brendan Gregg — perf Examples	The practical reference for Linux performance counters. Bookmark it.
Agner Fog — Optimizing software in C++	Chapter 2 covers benchmarking methodology properly.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Build the benchmarking harness from §3.1. Verify it by timing an empty function and a function with a known cost, and confirm the overhead is small relative to what you intend to measure.
- 2 Write a loop whose result is unused and confirm your compiler deletes it entirely at -O2. Inspect the assembly on Compiler Explorer to prove it. Then add a consume barrier and confirm the work returns.
- 3 Measure the same kernel in blocks (AAABBB) and interleaved (ABABAB) under sustained load. Log CPU frequency throughout. Quantify the thermal bias.
- 4 Compute the Amdahl speedup for your own code: profile a program you have written, identify the top function by time, and calculate the maximum possible gain from optimising it perfectly.
- 5 Run perf stat (or VTune) on three programs: a tight arithmetic loop, a random-access memory loop, and a branchy loop. Record IPC, cache miss rate, and branch miss rate for each, and explain the pattern.
- 6 Reproduce the latency table of §5 for your own machine as far as you can with simple timing. You will complete this properly in Module 05.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the performance equation and say what determines each factor.
- 2 Why is CPI a property of a program-processor pair rather than of the processor alone?
- 3 Give two reasons MIPS is a bad metric.
- 4 If an optimisation makes 80% of a program twice as fast, what is the overall speedup? What if that 80% became instantaneous?
- 5 Name four ways a benchmark can measure something other than what was intended.
- 6 Why take the minimum of several trials rather than the mean?
- 7 How many times slower is a main-memory access than an L1 hit, and what does the RAM model charge for each?